The Documentation Duality standard and three-tier skill architecture represent an empirical bet: that structured, machine-readable documentation measurably improves AI agent performance in research codebases. This section reports our key observations.

The documentation investment creates a positive feedback loop: as agents produce higher-quality outputs from structured context, developers maintain that documentation, which in turn improves future interactions (Lau and Guo 2025). We observed this concretely during `template/` development—capability-specific `SKILL.md` descriptors were refined through iterative AI-assisted generation, serving as both input prompt and output validator.

The `SKILL.md` layer, with its MCP-aligned YAML frontmatter (Anthropic 2024), provides a bridge to the agentic software paradigm. Lu et al.'s AI Scientist (Lu et al. 2024) demonstrates end-to-end autonomous research, while OpenHands is evaluated on SWE-Bench Verified (Wang et al. 2024; Jimenez et al. 2024). These systems motivate structured, protocol-aligned tool inventories for unfamiliar codebases. An agent navigating `template/` reads root `AGENTS.md` and `.agents/AGENTS.md` for shared constraints and generic workflows, scans `AGENTS.md` for local contracts, and can invoke skill-enabled capabilities through `SKILL.md` descriptors. The live module inventory verifies `AGENTS.md` and `README.md` coverage; `SKILL.md` remains a capability-specific layer rather than a universal claim about every infrastructure directory.

This three-tier model is, to our knowledge, novel in the research software engineering literature. The scale of the investment is substantial: 380 Markdown files under `docs/` alone, plus an `AGENTS.md`/`README.md` pair in documented directories and a `SKILL.md` descriptor on skill-enabled infrastructure modules. That count is itself injected from live introspection—the manuscript refuses to quote a documentation total it cannot recompute—and it represents a deliberate commitment to machine-readable context that shrinks the surface on which an agent can hallucinate.

The Learning Curve I

The Thin Orchestrator pattern imposes a cognitive overhead on researchers accustomed to writing monolithic scripts. The requirement to factor all logic into `src/` modules and use scripts only as stateless wiring introduces an additional layer of indirection. We mitigate this through:

The Learning Curve I

1. **Template exemplars**: `template_code_project` ships minimal optimization commentary; `template_prose_project` and `template_aut oresearch_project` broaden narrative + retrieval scaffolding.
2. **Documentation Duality**: Every directory has both `README.md` (for humans) and `AGENTS.md` (for AI collaborators), reducing the cost of navigation.
3. **Interactive orchestrator**: `run.sh` provides a TUI menu that abstracts pipeline complexity.
4. **Skill-level documentation**: The `docs/guides/` directory provides a progressive sequence of beginner, intermediate, advanced, and expert guides alongside a comprehensive new-project setup checklist.

Limitations I

- **LaTeX dependency**: The rendering pipeline requires a full TeX distribution (TeX Live or MiKTeX), which is a 4–6 GB install. This is the largest single dependency and is a barrier for researchers without system-level package management access.
- **Python-centric**: The infrastructure layer is Python-only. Projects in other languages can use the rendering and steganography stages but cannot leverage the `scientific` or `validation` modules.
- **Single-machine**: The pipeline runs locally. Distributed execution (e.g., across a compute cluster) is not natively supported, a gap where Snakemake, Nextflow, and CWL have clear superiority.
- **Steganographic robustness**: Alpha-channel overlays are stripped by aggressive PDF optimization tools (e.g., `qpdf --optimize`). QR codes are visible and removable. The current system provides *tamper detection* (via SHA-256 hashing) but not *non-repudiation* in the cryptographic sense—it lacks private-key digital signatures. An attacker with access to the source code could reproduce the watermark without having run the original pipeline.
- **Test duration**: The Zero-Mock policy increases test execution time from sub-second (mocked) to multi-minute (real) for the full infrastructure suite. This is acceptable for research workflows but may not suit continuous deployment scenarios.

Limitations II

- **AI-native writing tools**: `template/` does not include an integrated AI writing assistant comparable to Overleaf's Copilot features or OpenAI Prism's GPT-5.2 context-aware editing. The `infrastructure.llm` module provides LLM review as a pipeline stage but not as an interactive writing environment.